

Full Length Article

Enhanced accuracy in first-spike coding using current-based adaptive LIF neuron

Siying Liu^{*,} Pier Luigi Dragotti^{*}

Communications and Signal Processing Group, Department of Electrical and Electronic Engineering, Imperial College London, London, SW7 2AZ, United Kingdom

ARTICLE INFO

Keywords:

Spiking neural networks
First-spike coding
Neural dynamics
Adaptive neuron model
Event-based data

ABSTRACT

First spike timings are crucial for decision-making in spiking neural networks (SNNs). A recently introduced first-spike (FS) coding method demonstrates comparable accuracy to firing-rate (FR) coding in processing complex temporal information through supervised learning. However, its performance still falls behind advanced approaches. In order to explore the potential of FS coding, we enhance the capability of SNNs in classifying auditory datasets by improving neural dynamics. We propose a current-based adaptive LIF neuron (CuAdLIF) with delayed responses and membrane potential adaptation to enhance temporal correlations and preserve long short-memory. Furthermore, we introduce strategies to minimize delays in decision-making and enable adaptive training for FS coding. Results show that the CuAdLIF neuron enhances the extraction of temporal features and significantly improves FS coding accuracy. In addition, our strategies effectively reduce output time delays.

1. Introduction

Spiking neural networks (SNNs) play a critical role in brain-inspired computing (Li et al., 2023). They aim to develop efficient and effective computational models that emulate biological neural systems. Unlike traditional artificial neural networks (ANNs), SNNs convey information through spikes (Pfeiffer & Pfeil, 2018), incorporating a temporal dimension to information processing in an energy-efficient way. The emergence of event-driven neuromorphic hardware and sensors has provided additional momentum to the advancement of SNNs (Anumula et al., 2018). SNNs are inherently suited to process asynchronous data sequences from neuromorphic sensors and for the implementation on neuromorphic hardware (Li et al., 2023). However, SNNs usually exhibit performance that do not match those of ANNs. Therefore, the investigation of leveraging temporal features in events to enhance performance through the dynamics at the neuron and network levels has become a critical research focus.

Researchers have proposed different neuron models to simulate diverse neuronal dynamics in a biologically plausible way while simplifying mathematical formulations, such as Hodgkin–Huxley model (Hodgkin & Huxley, 1952), spike response model (Gerstner, 1995), leaky integrate-and-fire (LIF) model (Hunsberger & Eliasmith, 2015), and current-based LIF (CuBa-LIF) neuron. However, simplified neural models often struggle to deliver high performance. Therefore, many studies have focused on enhancing feature representation at the levels of neurons, dendrites, and networks. Research has demonstrated

that incorporating heterogeneity in neuron parameters and temporal dynamics enhances the performance of SNNs. Methods introduced to achieve this include learning time constants (Fang et al., 2021; Perez-Nieves et al., 2021) and adaptive thresholds (Bellec et al., 2018; Chen et al., 2022; Shaban et al., 2021; Yin et al., 2021). Recently, the LIF model with temporal dendritic heterogeneity (Zheng et al., 2024) has shown superior performance in learning multi-scale dynamics. Due to the importance of spike timing in temporal features, recent advancements involve learning synaptic (Hammouamri et al., 2023) and axonal delays (Deckers et al., 2024; Grimaldi & Perrinet, 2023; Sun et al., 2023, 2024, 2022) for tasks with complex temporal structures. At the network level, incorporating recurrent connections has proven effective in enhancing temporal correlations and overall performance (Bittar & Garner, 2022; Shaban et al., 2021). Inspired by attention mechanisms in ANNs, models with attention modules (Yao et al., 2021, 2023; Yu et al., 2022; Zhu et al., 2022) and transformer-based networks (Zhou et al., 2023, 2022) have achieved impressive results on visual datasets. However, these networks are usually large and complex, making them more suitable for processing data sequences with low temporal resolution. Moreover, temporal-wise attention is not practical for sequential data processing. Another category of methods introduces adaptation variables in neural dynamics to enhance long short-term memory in neuron (Song et al., 2024; Yao et al., 2023a; Zhang et al., 2023). For example, Bittar and Garner (2022)

* Correspondence to: Center for Brain Inspired Computing Research, Department of Precision Instrument, Tsinghua University, Beijing, 100084, China.
E-mail address: siying-liu@tsinghua.edu.cn (S. Liu).

proposed an adaptive LIF (AdLIF) neuron, which simulates the sub-threshold membrane potential regime (Brunel et al., 2003; Engel et al., 2008; Izhikevich, 2001) and spike-triggered adaptation. The AdLIF neuron has demonstrated improved performance in speech recognition tasks (Deckers et al., 2024; Liu et al., 2023). However, most research has focused on firing-rate coding, and there is a lack of studies investigating the use of various neural models with other coding methods.

In the context of information encoding, SNNs employ two distinct approaches: rate coding and temporal coding (Brette, 2015; Kiselev, 2016). Rate coding transmits information through spikes over a certain time window, potentially overlooking temporal information. On the other hand, temporal coding directly leverages spike timings, but the learning scheme is more complicated and limits its performance. Supervised learning of rate coding usually involves surrogate gradients of binary spikes with respect to membrane potential (Shrestha & Song, 2017; Wu et al., 2018; Zenke & Ganguli, 2018), while temporal coding methods explore derivatives of postsynaptic spike times with respect to membrane potential. Among temporal coding strategies, time-to-first-spike (TTFS) methods (Comsa et al., 2021; Göltz et al., 2021; Kheradpisheh & Masquelier, 2020; Mirsadeghi et al., 2021; Mostafa, 2018; Sakemi et al., 2023; Zhou et al., 2021) utilize the first spike time in decision-making. However, these methods assume each neuron fires only once, which is not realistic to process real-world event data sequences.

Recent study (Liu et al., 2023) proposed a decision-making strategy based on first-spike (FS) coding, which encodes the FS timings of output neurons. FS coding does not impose restrictions on the number of spikes for each neuron, demonstrating effectiveness on data sequences with rich temporal structures by focusing on the first spike. However, its performance is constrained when using only the current-based LIF (Cuba-LIF) neuron model. Additionally, FS coding typically exhibits longer time delays compared to FR coding and requires the careful selection of hyperparameters. Therefore, we aim to further explore the potential of FS coding by improving neural dynamics and enhancing training strategies. We extend the AdLIF neuron to a current-based AdLIF (CuAdLIF) neuron by incorporating another equation for the input signal. This adaptation introduces delays and diverse temporal dynamics, thereby effectively enhancing the representation of temporal features. In addition to this novel neuron model, we enhance FS coding by implementing strategies for time constant initialization to reduce decision-making delays and incorporating adaptive window sizes in backpropagation for more flexible training.

In the experiments, we address the classification problems posed by event data sequences with rich temporal structures. Specifically, we evaluate models with different neuron types and architectures on realistic speech recognition datasets, namely, SHD, SSC (Cramer et al., 2022), and N-TIDIGITS (Anumula et al., 2018). We explore FS coding across various neuron models and analyze their responses to changing signals. Our CuAdLIF neuron effectively enhances the representation of temporal features and improves classification accuracy. Furthermore, the results highlight the potential of FS coding to achieve high accuracy in classifying data with complex temporal structures.

The main contributions of this paper include:

1. Introducing the CuAdLIF neuron model to improve FS coding by incorporating delayed responses and diverse neuronal activities, which effectively enhances temporal correlation and maintains long-term memory.
2. Enhancing FS coding through strategies that facilitate adaptive training and minimize decision-making delays.
3. Performing an in-depth analysis of FS coding across various neuron types, demonstrating its adaptability and superior performance in classifying data sequences with complex temporal structures. Our method achieves state-of-the-art results when using only the first spike for decision making.

2. Methodology

2.1. Current-based adaptive LIF neuron

Leaky integrate-and-fire (LIF) neuron is the most widely used simplified neuron model. However, it cannot simulate various neuronal activities in biological systems. Some recent studies simulate subthreshold membrane potential oscillations in the neuron model by linearly coupling a adaptation variable to the membrane potential, such as the adaptive LIF (AdLIF) neuron (Bittar & Garner, 2022). Inspired by AdLIF neuron, we propose a current-based adaptive LIF (CuAdLIF) neuron by incorporating low-pass temporal filtering for input signals, similar to the current-based LIF (CuBa-LIF) neuron. Unlike the AdLIF neuron, which exhibits instant responses to input stimuli, the CuAdLIF neuron introduces a time delay, thereby enriching temporal dynamics of the membrane potential.

Consider a set of presynaptic neurons $j = 1, 2, \dots, J$ connected to a postsynaptic neuron i , the dynamics of the CuAdLIF neuron model are as follows:

$$\begin{aligned} \tau_s \frac{dI_i(t)}{dt} &= -I_i(t) + \sum_j w_{ij} \sum_{t_{j,m} < t} \delta(t - t_{j,m}), \\ \tau_m \frac{dU_i(t)}{dt} &= -(U_i(t) - U_r) + I_i(t) - W_i(t) - \tau_m \theta \sum_{t_i^f < t} \delta(t - t_i^f), \\ \tau_w \frac{dW_i(t)}{dt} &= -W_i(t) + a(U_i(t) - U_r) + \tau_w b \sum_{t_i^f < t} \delta(t - t_i^f) \end{aligned} \quad (1)$$

where τ_m , τ_s and τ_w are the time constant of membrane potential $U(t)$, synaptic current $I(t)$ and adaptation variable $W(t)$ respectively, and $\delta(t)$ is the Dirac function representing a spike function. Here $t_{j,m} < t$ is the firing time of the m th spike generated by the j th presynaptic neuron, and $t_i^f < t$ represents all the previous spike times of the postsynaptic neuron i . Moreover, the synaptic weight between neuron i and j is denoted as w_{ij} . The neuron fires a spike once the membrane potential $U(t)$ reaches a threshold θ . The membrane potential is then reset to resting potential U_r , where we set $U_r = 0$.

Compared with CuBa-LIF neuron, the adaptation variable $W(t)$ incorporates a linear term that relates to the membrane potential $U(t)$ and a nonlinear term activated by spikes. When no spikes occur, the variable $W(t)$ dynamically changes with the membrane potential, regulated by parameters a and τ_w . This process induces subthreshold oscillations in $U(t)$, as illustrated in Fig. 1. On the other hand, when a spike fires, $W(t)$ is increased by a spike with a value of b , denoted as $b \sum_{t_{i,n} < t} \delta(t - t_{i,n})$ in Eq. (1). This spike-triggered term, particularly with its nonlinearity and conditional logic via the threshold-based spike function, leads to increased diversity in neural dynamics, as shown in Fig. 2.

Specifically, Fig. 1 illustrates the dynamics of AdLIF and CuAdLIF neurons when no spike fires, with τ_w set to 100 time steps in alignment with the configuration specified in Bittar and Garner (2022). In this case, the spike-triggered term with b does not contribute to the neural dynamics. With varying values of a , both neurons exhibit distinct behaviors. When $a = 0$, the postsynaptic potential of AdLIF neuron follows that of a LIF neuron, while the CuAdLIF neuron exhibits dynamics of a CuBa-LIF neuron. When receiving a stimulus at step 0, the AdLIF neuron gives an instant response, whereas there is a delayed evolution of stimuli for the CuAdLIF neuron. Subsequently, the potential decays with oscillations, eventually converging around 0. The value of a determines the speed of this convergence and the frequency of oscillations.

In Fig. 2, the firing of a spike and membrane potential results in a joint impact on neuronal activities. This effect leads to diverse subthreshold oscillations using different combinations of a and b , playing a role in the preservation of long-term memory for stimuli through temporal correlations.

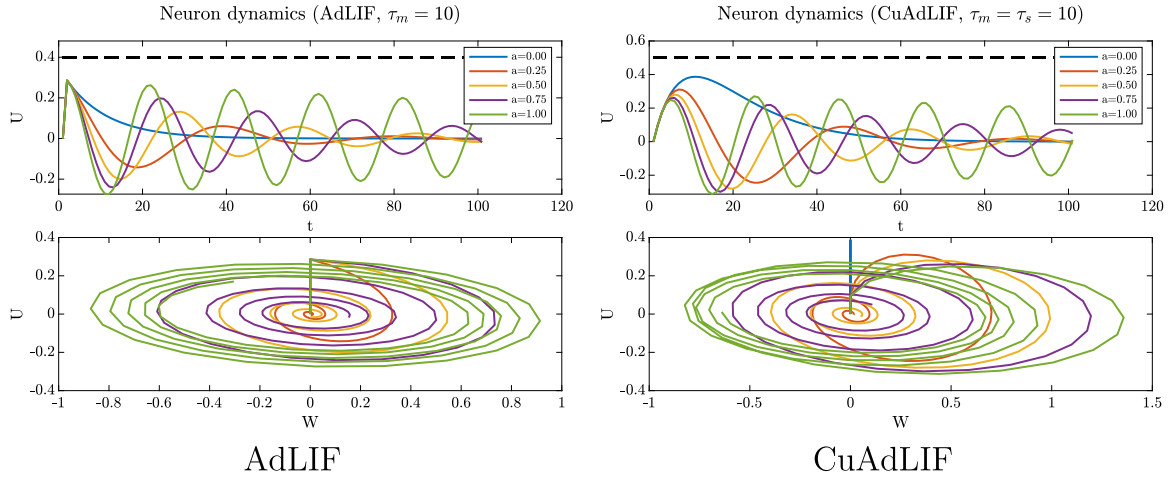


Fig. 1. Dynamics of AdLIF and CuAdLIF neurons with varying a when no spike fires ($\tau_w = 100, a \in [0, 1]$). In this case, the term with b does not contribute to the neural dynamics. The upper U - t plots reveal that increasing a intensifies oscillations, while the lower U - W plots show a longer convergence time with higher a . When $a = 0$, the membrane potential U of the AdLIF neuron behaves like that of a LIF neuron with immediate response, whereas the CuAdLIF neuron exhibits dynamics akin to a CuBa-LIF neuron with filtered input current.

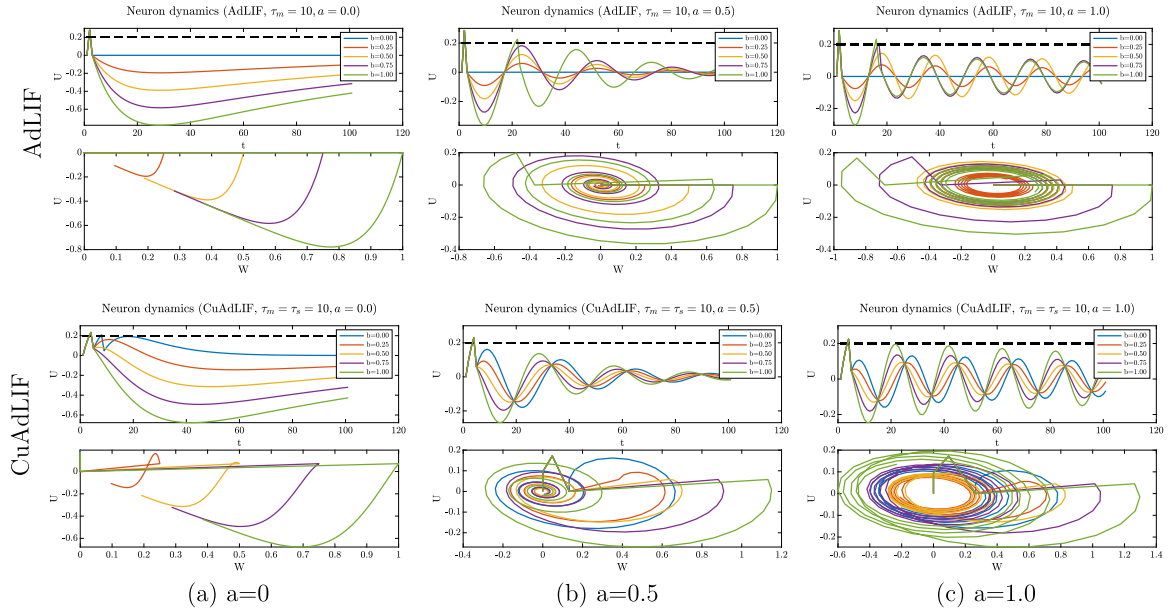


Fig. 2. Dynamics of AdLIF and CuAdLIF neurons with varying a, b after firing a spike ($\tau_w = 100, a, b \in [0, 1]$). In each plot, we use a constant value for a , while utilizing different colors to represent the distinct responses achieved by varying the value of b . Distinct oscillation (upper plots) and convergence (lower plots) patterns are observed for AdLIF and CuAdLIF neurons. The joint effect of a and b introduces diverse subthreshold oscillations to the neural dynamics, leading to long-term temporal correlations and distinct firing patterns.

The time constant τ_w of the adaptation variable is typically significantly longer than τ_s and τ_m , which implies a long-term impact on neuron activities. Note that we constrain both a and b in the range of $[0, 1]$. This differs from the settings in Bittar and Garner (2022), where a and b are limited to $a \in [-1, 1]$ and $b \in [0, 2]$. Our observations indicate that when $a < 0$, the postsynaptic potential experiences divergence, and an excessively large b induces pronounced oscillations with significantly higher amplitudes, leading to consistent firing. Therefore, our chosen constraints contribute to reasonable decays in membrane potential and promotes more stable training. A detailed analysis of how the system stability is affected by the value of a and b is presented in Appendix.

2.2. First-spike coding

In the context of a classification problem with event-based data, we aim to make decisions for an event stream $\mathcal{E}$ within a time window of length ΔT . For computational convenience, we typically divide the

event sequence into T time steps with a resolution of Δt , yielding a decision window $\Delta T = T\Delta t$. In Section 2.2.1, we introduce the discrete time formulation for CuAdLIF neuron with first-spike coding. In Section 2.2.2, we explain how CuBa-LIF neurons with large time constants and thresholds should be used to achieve first-spike coding and propose a strategy to reduce decision-making time delays.

2.2.1. Discrete time formulation with first-spike coding

The CuAdLIF neuron model is discretized in time to construct a multi-layer SNN. Given an SNN model with L layers, the membrane potential $U^l[n]$ is evolved through layers $l = 1, 2, \dots, L$ and time steps $n = 1, 2, \dots, T$. When the membrane potential of neuron i in layer l at time step n is greater than a threshold: $U_i^l[n] \geq \theta$, a spike is generated and is denoted as $S_i^l[n] = 1$. Otherwise $S_i^l[n] = 0$.

The input signal for the i th neuron in the l th layer at step n is a joint stimulus derived from spikes generated by neurons in the previous layer and, optionally, spikes generated by neurons within the same layer

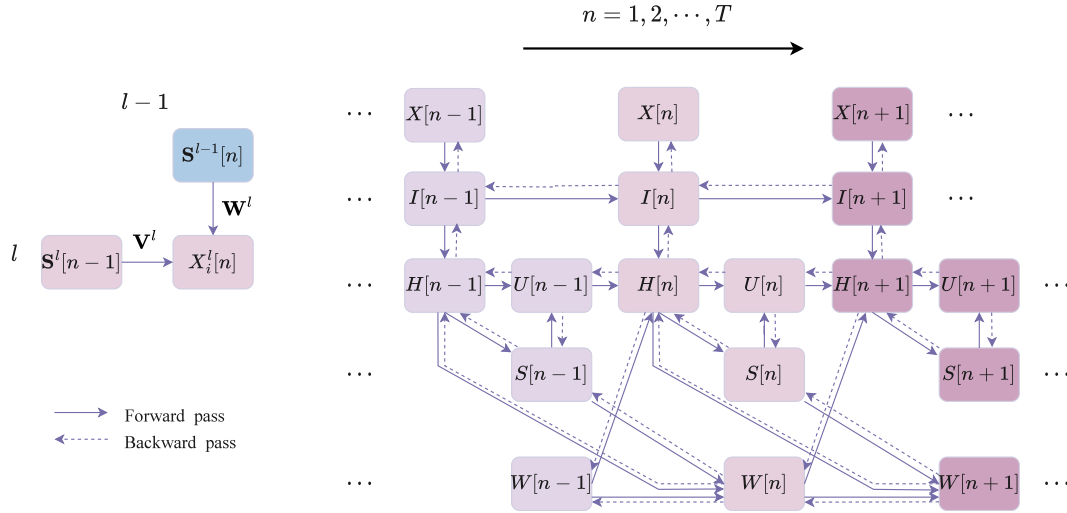


Fig. 3. Forward and backward propagation of a single neuron i in layer l . The input signal $X[n]$ is determined by the spikes from the last layer $l-1$ and spikes of previous step $n-1$ if recurrent connections are used.

from the previous step when recurrent connections are used. This input $X_i^l[n+1]$ can be expressed as:

$$X_i^l[n] = \sum_{j=1}^{Q(l-1)} w_{ij}^l S_j^{l-1}[n] + \sum_{k=1}^{Q(l)} v_{ik}^l S_k^l[n-1]. \quad (2)$$

The first term on the right side represents the weighted summation of presynaptic spikes in Eq. (1). The last term denotes the impact of recurrent connections, where v_{ik} is the weight of a recurrent connection between the k th and i th neuron within the same layer l . The number of neurons in the l th layer is denoted as $Q(l)$. For simplification, we denote $X_i^l[n]$ as $X[n]$, and similarly for all variables related to the i th neuron in layer l , such as $I[n]$ and $U[n]$, throughout the rest of the thesis (see Fig. 3).

Using the exponential decay method and according to Eq. (1), the update of synaptic current $I[n]$, membrane potential $U[n]$, adaptation variable $W[n]$ and spike firing from step $n-1$ to n are as shown in Fig. 3 and as follows:

$$\begin{aligned} I[n] &= \beta I[n-1] + X[n] \\ H[n] &= \alpha U[n-1] + (1-\alpha)(I[n] - W[n-1]) \\ S[n] &= f(H[n] - \theta) \\ U[n] &= H[n](1 - S[n]) \\ W[n] &= \gamma W[n-1] + aH[n-1] + bS[n-1] \end{aligned} \quad (3)$$

where the membrane potential before and after firing a spike is denoted as $H[n]$ and $U[n]$ respectively. Specifically, the membrane potential $H[n]$ is reset by multiplying it with $(1-S[n])$, which has a similar effect as reducing it by the threshold in Eq. (1). Moreover, $\alpha = e^{-\frac{\Delta t}{\tau_m}}$, $\beta = e^{-\frac{\Delta t}{\tau_s}}$ and $\gamma = e^{-\frac{\Delta t}{\tau_w}}$ are forgetting factors. All the parameters related to time constants, α , β , and γ , as well as the coefficients a and b , are trainable. Finally, the spiking process can be described as a step function of the membrane potential:

$$S[n] = f(H[n]) = \begin{cases} 1, & H[n] \geq \theta, \\ 0, & H[n] < \theta. \end{cases} \quad (4)$$

The output spike trains of the network are encoded into discrete spike times in order to implement first-spike (FS) coding. Building on our prior work (Liu et al., 2023), the output time of the i th neuron at step n is given by:

$$t_i^L[n] = h(S_i^L[n]) = \begin{cases} n\Delta t, & S_i^L[n] = 1, \\ t_i^{inf}, & S_i^L[n] = 0, \end{cases} \quad (5)$$

where

$$t_i^{inf} = (T+1)\Delta t. \quad (6)$$

Spikes are encoded into discrete timestamps, whereas other steps without spike firing are encoded with t_i^{inf} . Finally, the time of the first spike from the i th output neuron, denoted as t_i^F , is given by the minimum of the temporal codes:

$$t_i^F = \min_n(t_i^L[n]). \quad (7)$$

2.2.2. CuBa-LIF neuron for decision making

For decision-making based on the FS timing, it is important to highlight that the CuBa-LIF neuron is specifically employed in the output layer, while various neuron types can be employed in hidden layers to extract features. The reason is that a longer delay in the output first spike promotes better performance based on our experiments. However, the fast dynamics of neurons like AdLIF or CuAdLIF generally lead to very early spike firing. In contrast, the CuBa-LIF neuron can ensure that the membrane potential reaches the threshold and fires the first spike after sufficient information has been accumulated, particularly when utilizing large time constants. The discretized dynamics of CuBa-LIF neuron is similar to Eq. (3), excluding terms related to the adaptation variable $W[n]$. This configuration results in a hybrid SNN that integrates CuBa-LIF with other neuron types.

In addition, FS coding requires different hyperparameter configurations for feature extraction and decision making. According to Liu et al. (2023), achieving higher accuracy in FS coding necessitates larger time constants and thresholds in the output layer, resulting in longer time delays. Typically, the time constants for membrane potential τ_m and synaptic current τ_s are initialized to equal values. We denote the time constants and thresholds for hidden layers and the output layer as τ_1/θ_1 and τ_2/θ_2 , respectively. However, experiments in Liu et al. (2023) have demonstrated that a large $\tau_{m2} = \tau_{s2} = \tau_2$ significantly increase delay in decision making. Therefore, we aim to reduce time delay in FS coding by using different values for τ_{m2} and τ_{s2} in the output layer. For CuBa-LIF neurons, τ_{m2} primarily affects the response time to input stimuli, while τ_{s2} governs the decay speed of synaptic current. In other words, decreasing τ_{m2} can reduce response time, while maintaining a relatively large τ_{s2} can preserve the memory of previous stimuli. Therefore, we propose using a combination of a small τ_{m2} and a large τ_{s2} to reduce time delay in decision making while maintaining relatively high accuracy. Related experiments will be discussed in Section 3.5.

2.3. Backpropagation through surrogate gradients

We train our FS-based SNN in a supervised manner. A cross-entropy loss is used based on the FS times of the output neurons. This allows

us to minimize the FS time of the target neuron and maximize that of non-target neurons:

$$\mathcal{L}_{FS} = - \sum_{i=1}^C y_i \log \frac{\exp(-p_0 t_i^F)}{\sum_{j=1}^C \exp(-p_0 t_j^F)} + \lambda_t \sum_{\{i|t_i^F\} T \Delta t} y_i [\exp(q_0 t_i^F) - 1], \quad (8)$$

where y_i is the target label (0 or 1) of the i th class, C is the number of output neurons (classes), and p_0, q_0, λ_t are constant coefficients, where p_0 is used to control the speed of training and prevent the exponential function from taking an excessively high value. The second term is a constraint to penalize a target neuron which never fires. For ease of notation, we use $\mathcal{L}$ to represent $\mathcal{L}_{FS}$ in the rest of the paper.

The main challenge of supervised learning is how the error of the output FS timing propagates to spikes and the non-differentiability of the spike function with respect to membrane potential. Here we use surrogate gradient method to achieve backpropagation through time (BPTT).

2.3.1. Error assignment from FS time to spike

In the last layer, the error of the output FS timing is required to be propagated to related spikes:

$$\frac{\partial \mathcal{L}}{\partial S_i^L[n]} = \frac{\partial \mathcal{L}}{\partial t_i^F} \sum_{m=1}^T \frac{\partial t_i^F}{\partial t_i^L[m]} \frac{\partial t_i^L[m]}{\partial S_i^L[n]}. \quad (9)$$

Here we use the method in Liu et al. (2023) to achieve error assignment from FS timings to related spikes. The derivative $\frac{\partial \mathcal{L}}{\partial t_i^F}$ is used to distribute the error of the predicted time $\frac{\partial \mathcal{L}}{\partial t_i^F}$ across all steps. For active neurons, the error is only related to the first spike time, so $\frac{\partial t_i^F}{\partial t_i^L[n]}$ is equal to 1 only when $t_i^L[n] = t_i^F$. However, to promote the weight update for inactive neurons, we assign the error $\frac{\partial \mathcal{L}}{\partial t_i^F}$ to all steps, making $\frac{\partial t_i^F}{\partial t_i^L[n]}$ always equal to 1.

For the second gradient, a negative Gaussian window is used to distribute the error of time at step m to the spikes around it:

$$\frac{\partial t_i^L[m]}{\partial S_i^L[n]} = -\frac{A}{2\pi\sigma} e^{-\frac{(m-n)^2}{2\sigma^2}}, \quad (10)$$

where $A > 0$ is the amplitude, and $\sigma = \lfloor \frac{T}{D} \rfloor$ is the standard deviation, determined by the length of the sequence T and a constant factor D . The width of Gaussian window W_G is the minimum of 3-sigma limit and the length of whole sequence T , i.e., $W_G = \min\{6\sigma + 1, T\}$.

In this work, we propose to use an adaptive D to dynamically adjust Gaussian window size during training. At the initial stage, a small value for D (corresponding to a large σ) is utilized to create a large window size, facilitating the propagation of error across multiple steps and the updating of all involved weights. During training, the value of D decreases to reduce the window size. Consequently, the error assignment becomes more concentrated on the time steps surrounding the first spike, giving priority to the impact of the first spike. Specifically, the value of D at current epoch is as follows:

$$D = D_0 \left(\frac{D_{\max}}{D_0} \right)^{\frac{\text{epoch}}{\text{max_epoch}}}, \quad (11)$$

where $D_0 = 2$, $D_{\max} = \frac{T}{2}$. The effectiveness of adaptive D is validated in Section 3.6.

2.3.2. Surrogate gradient descent learning through spikes

After propagating error from FS times to spikes in the output layer, the error can be propagated through spikes in the rest of the network. To solve the non-differentiability of the spike function, a surrogate gradient (Zenke & Ganguli, 2018) is used to estimate the derivative of postsynaptic spikes with respect to membrane potential. Specifically,

the gradient of the step function in Eq. (4) is estimated using a fast sigmoid function in the backward pass:

$$f'(U) \approx \frac{1}{(1 + \rho|U - \theta|)^2}. \quad (12)$$

With the error assignment from FS timings to spikes and surrogate gradients of spike function, the overall backpropagation pipeline can be constructed. In the output layer, the error derived from FS timings $\frac{\partial \mathcal{L}}{\partial S_i^L[n]}$ propagates to spikes and subsequently the spike error $\frac{\partial \mathcal{L}}{\partial S_i^L[n]}$ propagates across layers. According to the discretization described in Eq. (2)–(3), the error of adaptation term $\frac{\partial \mathcal{L}}{\partial W^l[n]}$, membrane potential $\frac{\partial \mathcal{L}}{\partial U^l[n]}$, synaptic and input signal $\frac{\partial \mathcal{L}}{\partial X^l[n]}$ can be calculated. Additionally, the error of trainable parameters $\alpha, \beta, \gamma, a, b$ can be determined.

Finally, to learn the weights $\mathbf{W} = \{w_{ij}^l\}$ and $\mathbf{V} = \{v_{ik}^l\}$, the derivative of error with respect to weights $\mathbf{W}^l, \mathbf{V}^l$ in each layer can be calculated through $\frac{\partial \mathcal{L}}{\partial X^l[n]}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \sum_{n=1}^T \frac{\partial \mathcal{L}}{\partial X^l[n]} \frac{\partial X^l[n]}{\partial \mathbf{W}^l} = \sum_{n=1}^T \frac{\partial \mathcal{L}}{\partial X^l[n]} S^{l-1}[n], \quad (13)$$

and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{V}^l} = \sum_{n=1}^T \frac{\partial \mathcal{L}}{\partial X^l[n]} \frac{\partial X^l[n]}{\partial \mathbf{V}^l} = \sum_{n=1}^T \frac{\partial \mathcal{L}}{\partial X^l[n]} S^l[n]. \quad (14)$$

3. Numerical results

3.1. Experimental settings

3.1.1. Datasets

Previous research (Liu et al., 2023) has shown that FS coding excels with sequences that exhibit rich temporal patterns, such as those found in auditory datasets, whereas it is less effective compared to FR coding for visual data sequences. To investigate how different neural dynamics influence the capture of temporal information, we use realistic audio datasets, including SHD, SSC (Cramer et al., 2022) and N-TIDIGITS (Anumula et al., 2018) datasets. The visualization of these datasets is shown in Fig. 4.

SHD and SSC dataset. Spiking Heidelberg Dataset (SHD) and Spiking Speech Command (SSC) (Cramer et al., 2022) are dynamic audio datasets generated using Lauscher, an artificial cochlea model. The SHD includes 20 classes of spoken digits from 0 to 9 in both German and English language. The SSC is based on the Speech Commands release by Google, comprising utterances from a large number of speakers in less controlled environments, including 35 word categories. This dataset is much more challenging due to the substantial noise within the sequences. As illustrated in Fig. 4, the patterns within the same class vary significantly, making the training more challenging. Each sequence in both datasets is around 1s.

N-TIDIGITS dataset. N-TIDIGITS Dataset (Anumula et al., 2018) transforms TIDIGITS dataset into spikes with the dynamic audio sensor, CochleaAMS1b. It includes 11 classes of spoken numbers 0–9 and a word “oh”. The length of sequences is around 0.08–2.46 s, but most of the sequences are less than 1.2 s.

3.1.2. Architecture and training details

We utilize standard SNN architectures with fully-connected (FC) layers, implemented based on SpikingJelly framework (Fang et al., 2023). Typically, a layer structure is Linear+BN+Neuron+DP. Each layer comprises a linear operation followed by batch normalization, neuronal operations, and dropout, with dropout exclusively applied at hidden layers. Architectures for specific datasets include 700-FC256-FC256-20 for SHD, 64-FC256-FC256-11 for N-TIDIGITS, and 700-FC512-FC512-35 for SSC. For example, the notation of the first architecture represents a network with an input size of 700 and 20 output classes, consisting of two FC layers with 256 neurons each, and outputting 20 classes. The

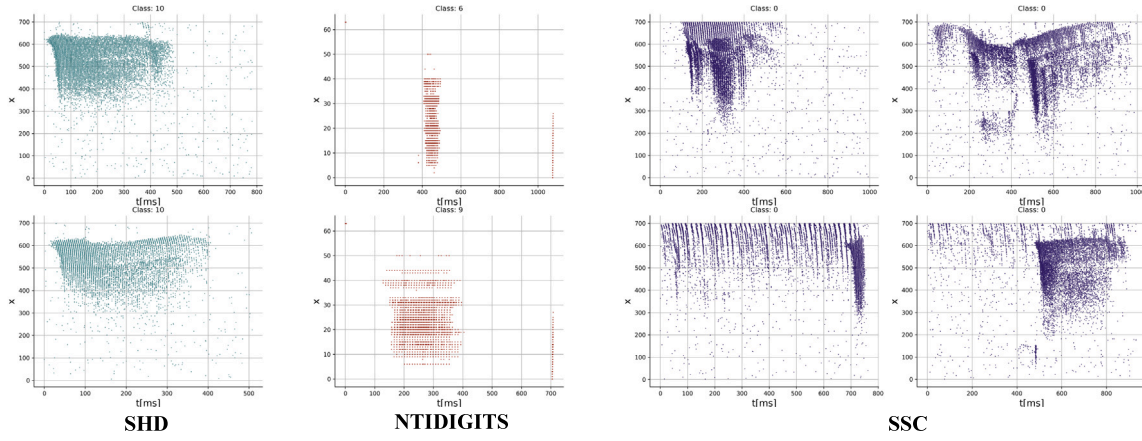


Fig. 4. Visualization of SHD, NTIDIGITS, and SSC datasets, with the x-axis representing event addresses x and the y-axis representing time steps t .

Table 1

Hyperparameters for different tasks in training. Adaptive D during training is denoted as adapt.

	Name	Notation	SHD	NTIDIGITS	SSC
Neuron	Time constant	τ_1/τ_2	5/60	5/40	3/20
	Threshold	θ_1/θ_2	1/10(CuAdLIF), 0.5/10(AdLIF), 2/5(PCuLIF/ PCuLIF-R)	2/10(CuAdLIF), 0.5/10(AdLIF), 5/10(PCuLIF/ PCuLIF-R)	1/10(CuAdLIF) 0.5/10(AdLIF) 1/10(PCuLIF/ PCuLIF-R)
Data	Time resolution	Δt	10 ms	5 ms	10 ms
	Time step	T/T_E	100/0	250/0	100/50
Loss	$\mathcal{L}_{FS}$ factor	p_0	0.2	0.1	0.2
	$\mathcal{L}_{FR}$ factor	p_1	20	20	20
	Penalize factors	λ/q_0	0.01/0.02	0.01/0.02	0.01/0.02
	Gauss. amplitude	A	200	500	300
	Gauss. inverse window size factor	D	16/adapt.	16/adapt.	64/adapt.
Training	Learning rate	η	$1e^{-3}$	$1e^{-3}$	$1e^{-2}$
	Epoch	N_{ep}	100	200	100
	Batch size	B	128	128	512

dropout rate is set to 0.25 for SHD and N-TIDIGITS datasets, while a dropout rate of 0.3 is utilized for the SSC model.

We test models with various types of neurons in hidden layers, including Parametric CuBa-LIF (PCuLIF), AdLIF and our CuAdLIF neuron. The PCuLIF neuron is essentially a CuBa-LIF neuron with trainable τ_m and τ_s . The model labeled as PCuLIF-R employs PCuLIF neurons and incorporates recurrent connections. These connections are added to the hidden layer without the use of dropout. We found that adding recurrent connections yields negligible benefits for AdLIF and CuAdLIF models. Thus we only present results of recurrent models for the PCuLIF neuron. As explained in Section 2.2.2, we exclusively alter the neuron types in hidden layers, while maintaining the use of standard CuBa-LIF neurons in the output layer. This configuration results in different time constants and thresholds for the hidden layers denoted as τ_1/θ_1 , and for the output layer, represented as τ_2/θ_2 . They are initialized to appropriate values and trained to achieve optimized performance.

During training, the input signals are discretized into T time steps with a temporal resolution Δt , and an empty sequence with a length of T_E is added to enhance accuracy and prevent inactive neurons which fire late (Liu et al., 2023). In addition, weights are initialized using Xavier uniform distribution, and the Adam optimizer is employed with a weight decay of $1e^{-4}$. Additionally, the gradients are clipped with a maximum norm of 10 during the training process. The parameter of surrogate gradient ρ is set to 5. The learning rate, number of epochs, and batch size are represented as η , N_{ep} , and B respectively. The specific configurations for each dataset are outlined in Table 1.

We typically train our models based on FS timings using the loss function $\mathcal{L}_{FS}$ in Eq. (8). For comparative analysis, we also evaluate results with models trained based on firing rate (FR) in Section 3.4. Similarly, a cross-entropy loss function maximizing the FR of the target neuron is used and given by:

$$\mathcal{L}_{FR} = - \sum_{i=1}^C y_i \log \frac{\exp(p_1 f_i)}{\sum_{j=1}^C \exp(p_1 f_j)}, \quad (15)$$

where p_1 is a constant value and $f_i = \frac{1}{T} \sum_{n=1}^T s_i^{L,n}$ is the spike rate within T steps of the i th neuron in the last layer L .

3.1.3. Evaluation

The results are compared based on accuracy, time delay, and spike count. Time delay and spike count are evaluated following the metrics outlined in Liu et al. (2023). Given the sequential data processing nature of the network, accuracy tends to increase with a longer time window. Consequently, we assess time delay as the time when the accuracy reaches 50% or 90% of its peak within the specified time window, denoted as $t_d(50\%)$ and $t_d(90\%)$ respectively.

Regarding spike count, we quantify it as the average number of spikes per neuron in the system, represented as N_s . Although the spike count varies across different neuron types, time constants, and thresholds, it serves as an indicator of power consumption to some extent.

Table 2

Comparison of results with other advanced methods. Note: (Avg., Best).

	Method	Acc (%)
SHD	Adaptive SRNN (Yin et al., 2021)	90.4
	TA-SNN (Yao et al., 2021)	91.08
	STSC-SNN (Yu et al., 2022)	92.36
	RadLIF (Bittar & Garner, 2022)	94.62
	LIF with learned delays (Hammouamri et al., 2023)	95.07 ± 0.24
	Adaptive axonal delay (Sun et al., 2023)	92.45
	AdLIF+synaptic delays (Deckers et al., 2024)	(95.02, 96.26)
	DH-SRNN (dendritic hetero.) (Zheng et al., 2024)	92.1
	Ours (CuAdLIF+FS)	(94.2, 95.0)
NTIDIGITS	GRU-RNN (Anumula et al., 2018)	90.9
	Phased-LSTM (Anumula et al., 2018)	91.25
	ST-RSBP (Zhang & Li, 2019)	93.63 ± 0.27
	SLAYER + spike-rate (Shrestha et al., 2022)	94.19 ± 0.18
	SLAYER+SpikeMax (Shrestha et al., 2022)	93.21 ± 0.32
	Adaptive axonal delay (Sun et al., 2023)	95.09
	Ours (CuAdLIF+FS)	(94, 94.53)
	Ours (CuAdLIF+FS)	(94, 94.53)
SSC	Adaptive SRNN (Yin et al., 2021)	74.2
	RadLIF (Bittar & Garner, 2022)	77.4
	LIF with learned delays (Hammouamri et al., 2023)	79.77 ± 0.09
	AdLIF+synaptic delays (Deckers et al., 2024)	79.81
	DH-SRNN (dendritic hetero.) (Zheng et al., 2024)	82.46
	Ours (CuAdLIF+FS)	(78.2, 78.7)
	Ours (CuAdLIF+FS)	(78.2, 78.7)

Table 3

Comparison results of different neurons. PCuLIF-R denotes the recurrent network using the PCuLIF neuron in hidden layers. **Bold** values represent the best results for the same dataset. The results are the average of 5 trials. CuAdLIF models achieve the highest accuracy with relatively fewer spikes, although they exhibit a longer time delay compared to AdLIF models.

		Acc (%)↑	N_s ↓	$t_d(50\%)$ ↓	$t_d(90\%)$ ↓
SHD	PCuLIF	86.8	6.1	51.4	69.8
	PCuLIF-R	87.1	5.3	50.8	67.8
	AdLIF	94.2	3.7	55.2	71.6
	CuAdLIF	94.2	7.1	56.2	71.0
	CuAdLIF	94.2	7.1	56.2	71.0
NTIDIGITS	PCuLIF	87.6	10.8	106.4	154.2
	PCuLIF-R	90.1	15.8	101.6	149.8
	AdLIF	92.0	4.2	85.0	128.0
	CuAdLIF	94.0	5.0	95.8	143.6
	CuAdLIF	94.0	5.0	95.8	143.6
SSC	PCuLIF	73.2	7.0	77.4	104.8
	AdLIF	77.1	3.8	76.2	101.6
	CuAdLIF	78.2	5.2	82.2	108.0
	CuAdLIF	78.2	5.2	82.2	108.0

3.2. Results overview

The comparison results with other state-of-the-art methods on SHD, NTIDIGITS, and SSC datasets are presented in Table 2. The majority of these methods are trained using FR, incorporating recurrent connections or delays. Our CuAdLIF models, based on FS coding, demonstrate comparable results to other methods without recurrent connections. To the best of our knowledge, this represents the best performance achieved using only the first spike for decision making.

3.3. Comparison of different neuron types

3.3.1. Quantitative results

We apply various neuron types in model configurations and train them based on FS timings. Comparison results across the SHD, NTIDIGITS, and SSC datasets are summarized in Table 3. The different neuron types employed in hidden layers include PCuLIF, AdLIF, and CuAdLIF, with the additional inclusion of recurrent connections in hidden layers denoted as PCuLIF-R. The PCuLIF-R models employ the same neuron type and architecture as used in our previous paper (Liu et al., 2023).

Comparing results in Table 3, it is evident that CuAdLIF models demonstrate the highest accuracy, with AdLIF models also achieving good performance. Conversely, PCuLIF and PCuLIF-R models exhibit

inferior performance across all datasets. The superior performance of CuAdLIF and AdLIF neurons derives from their diverse oscillations, which generate temporal correlations at various time scales to enhance feature expression and promote effective decision-making. In contrast, the learnable time constants of PCuLIF neurons primarily affect the speed of membrane potential evolution without changing the shape of the postsynaptic potential. This highlights the significance of oscillations in neural dynamics for effective temporal feature extraction.

In terms of spike count and time delay, AdLIF models produce the fewest spikes and exhibit the shortest time delay. This is due to the instant response of the LIF neuron to input signals, while the current-based neuron introduces a low-pass filtering effect, resulting in a delayed neural response. However, the delay in response contributes to higher accuracy for CuAdLIF models.

The inclusion of recurrent connections for PCuLIF models significantly improve accuracy, but the performance remains inferior to that of AdLIF and CuAdLIF models. While recurrent connections contribute to improved temporal correlation, they do not match the inherent dynamics of the adaptive neurons. Recurrent connections introduce the effect of previous spikes, which is similar to the spike-triggered adaptation term. However, their ability to produce long-term effects for input stimuli is inferior to temporal oscillations.

3.3.2. Response to changing signals

We further conduct a test to observe the response to changing input signals and generate raster plots of output spikes from different models on SHD and NTIDIGITS datasets, as shown in Fig. 5. In this experiment, the input signal evolves through temporal changes, transitioning from class 0 to the final class. The black line in Fig. 5 indicates the target class and its duration, while colored dots represent output spikes from each output neuron in response to varying signals. Spikes from the target neuron are highlighted for better visualization.

Comparing the raster plots for different neuron types, AdLIF and CuAdLIF models generate fewer spikes from non-target neurons. This observation implies a quicker response and higher accuracy compared to PCuLIF models when adapting to changing signals. In addition, PCuLIF-R model generates increased number of non-target spikes, thereby deteriorating the accuracy of classification during transitions.

Although the model exhibits significant delays in response to individual classes, the response to changing signals is generally fast if classifying correctly, as depicted in Fig. 5. Furthermore, despite being trained based on the first spike, the target neuron in these models is capable to consistently fire more spikes than non-target neurons.

3.3.3. Model complexity and training speed

The CuAdLIF neuron model introduces increased complexity through its delay response and membrane potential adaptability mechanisms. In addition to synaptic weights and batch normalization for standard FC architecture, each CuAdLIF neuron includes five trainable parameters: α (related to τ_m), β (related to τ_s), γ (related to τ_w) and parameters a and b that influence spike-triggered adaptation. In contrast, the AdLIF neuron model excludes β and only involves parameters α , η , a , and b , making a total of four parameters. For the PCuLIF neuron, only α and β are used. Consequently, for a layer with N neurons, the CuAdLIF model adds $5N$ parameters, while the AdLIF model adds $4N$, and the PCuLIF model adds $2N$. This is relatively modest compared to the N^2 parameters required for the connections between layers, indicating that while the CuAdLIF model increases complexity, the number of additional parameters and operations remains limited.

Regarding convergence speed, we show the training loss for PCuLIF, AdLIF, and CuAdLIF models on SHD and NTIDIGITS datasets in Fig. 6. The CuAdLIF model exhibits faster convergence than the AdLIF model. While PCuLIF models converge more quickly, they face challenges in generalization, resulting in lower performance on the test dataset (see Table 3).

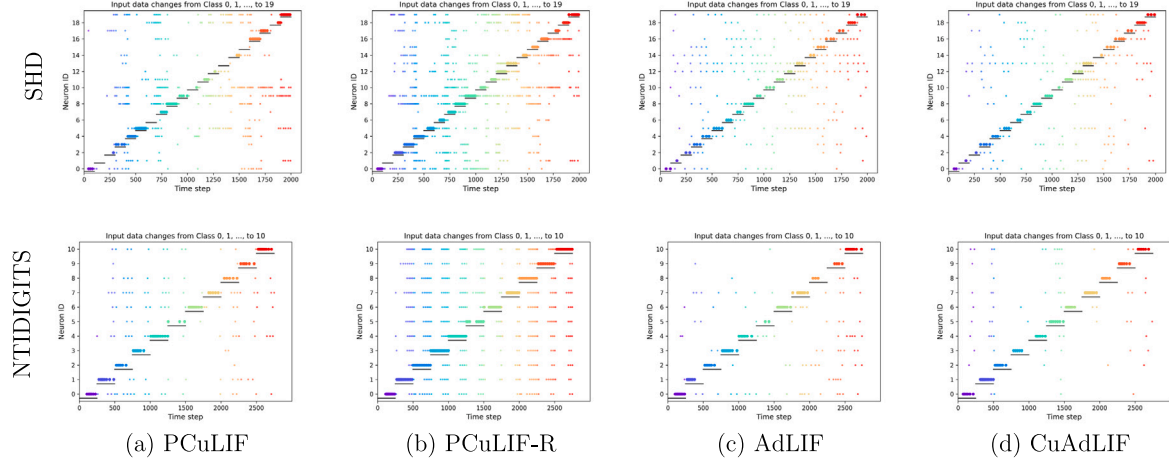


Fig. 5. Output spike raster plots of models with different neuron types for the SHD and NTIDIGITS dataset. Each figure illustrates the output spikes generated in response to varying input signals from class 0 to the last class. Colors represent output spikes from the same input signal, changing with the input signal. The black line indicates the target class and its duration, with spikes from the target neuron highlighted for better visualization. Fewer non-target spikes are observed for the AdLIF and CuAdLIF models, suggesting a faster response and higher accuracy in adapting to changing signals.

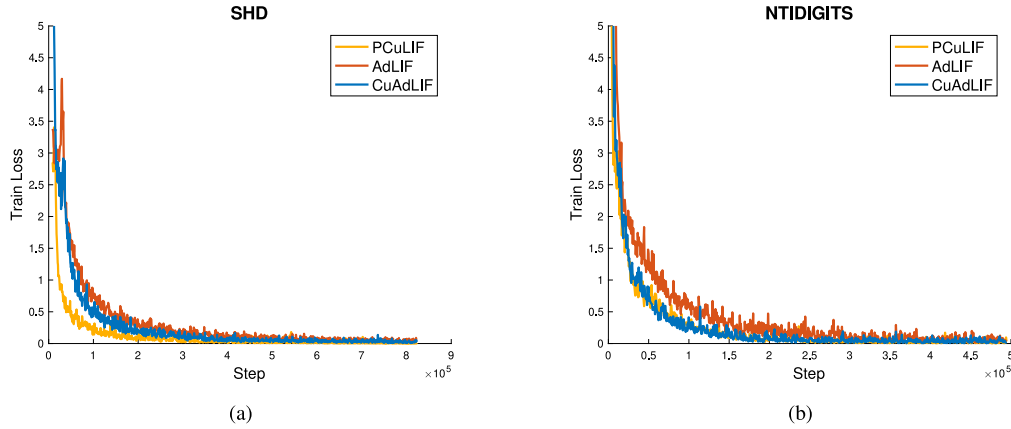


Fig. 6. Training loss for PCuLIF, AdLIF, and CuAdLIF models on SHD and NTIDIGITS datasets. The CuAdLIF model exhibits faster convergence compared to the AdLIF model.

Table 4

Comparison results of FS and FR models with AdLIF and CuAdLIF neurons. **Bold** values represent the best results for the same dataset. Acc (FS) and Acc (FR) denote evaluations based on FS coding and FR coding, respectively. The results are the average of 5 trials. FS models generally outperform FR models on these datasets. The CuAdLIF neuron shows a more significant improvement in FS coding than in FR coding when compared to the AdLIF neuron. In addition, FS models perform well in both FS and FR evaluations.

	Neuron	FS				FR			
		Acc (%) (FS)†	Acc (%) (FR)†	$N_s \downarrow$	$t_d(90\%) \downarrow$	Acc (%) (FS)†	Acc (%) (FR)†	$N_s \downarrow$	$t_d(90\%) \downarrow$
SHD	AdLIF	94.2	82.8	3.7	71.6	19.7	91.7	6.5	60.0
	CuAdLIF	94.2	86.3	7.1	71.0	8.5	92.3	6.4	59.0
NTIDIGITS	AdLIF	92.0	91.5	4.2	128.0	22.4	93.3	10.2	115.0
	CuAdLIF	94.0	93.3	5.0	143.6	23.3	93.6	11.6	121.0
SSC	AdLIF	77.1	73.8	3.8	101.6	5.4	75.2	7.0	79.2
	CuAdLIF	78.2	73.6	5.2	108.0	9.1	75.7	7.3	89.0

3.4. FR vs. FS models

We compare the performance of AdLIF and CuAdLIF models trained based on FS and FR respectively. When training based on FR, the architecture and parameters align with FS models. Table 4 presents related results, indicating that FS models generally outperform FR models on these datasets. Comparing the AdLIF and CuAdLIF models, the CuAdLIF neuron shows a more significant improvement in FS coding than in FR coding. The temporal delays introduced by current-based models make them particularly beneficial for FS coding over FR coding. In addition, all models are assessed with both coding schemes. We can

see that the FS models perform well in both FS and FR evaluations, whereas the FR models struggle with accurate classification using the first spike. In terms of time delay and spike count, FS models generate fewer spikes overall but experience longer time delays compared to FR models, suggesting that FS coding is more efficient to some extent.

Furthermore, FS models usually handle changing signals more effectively than FR models. Fig. 7 presents the output response to changing signals (from class 0 to the last class), depicting that FR models produce a significantly larger number of spikes compared to FS models. Although FR models respond quickly to input changes, the presence of non-target spikes introduces interference in the classification process.

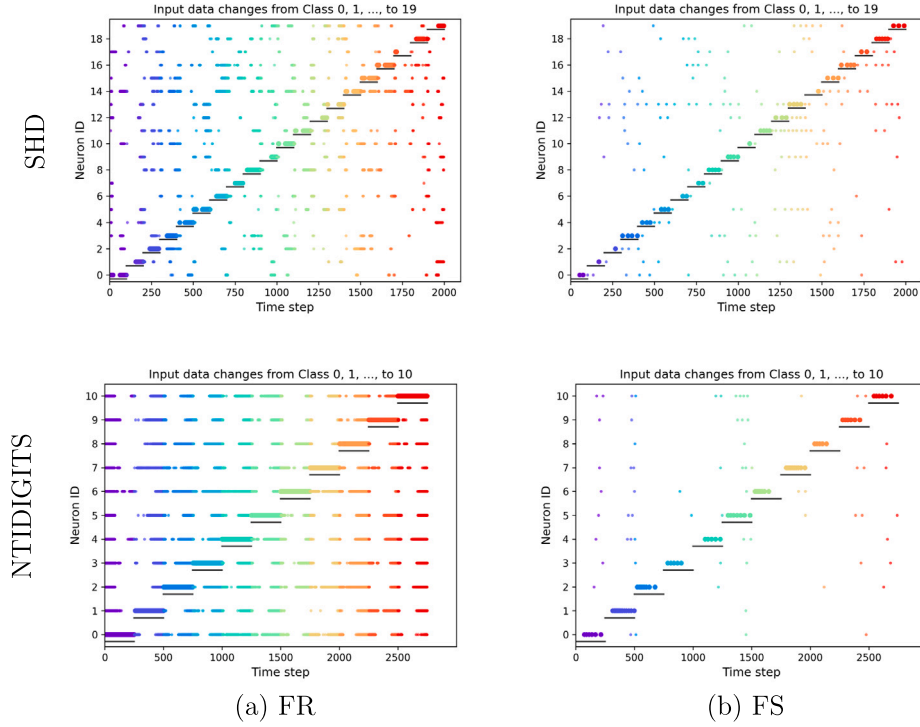


Fig. 7. Output spike raster plots of FR and FS models with CuAdLiF neuron for the SHD and NTIDIGITS dataset. Each figure illustrates the output spikes generated in response to varying input signals from class 0 to the last class. Colors represent output spikes from the same input signal, changing with the input signal. The black line indicates the target class and its duration, with spikes from the target neuron highlighted for better visualization. It is notable that the FR system encounter difficulties in accurately classifying signals that change over time.

Table 5

Comparison results of using different τ_{m2} and τ_{s2} in the output CuBa-LiF neuron. **Bold** values represent the best results for the same dataset. A smaller τ_{m2} leads to a significant reduction of time delay for FS models. Using a combination of a small τ_{m2} and a large τ_{s2} can effectively reduce time delay in FS coding without compromising accuracy.

	$\tau_{m2} - \tau_{s2}$	FR			FS		
		Acc (%) $\uparrow$	$t_d(50\%) \downarrow$	$t_d(90\%) \downarrow$	Acc (%) $\uparrow$	$t_d(50\%) \downarrow$	$t_d(90\%) \downarrow$
SHD	5-5	90.4	33.0	57.0	89.3	33.0	46.0
	5-20	93.4	32.0	55.0	92.9	40.0	54.0
	5-60	94.4	33.0	59.5	94.1	41.0	57.0
	60-60	92.3	35.0	59.0	94.2	56.2	71.0
NTIDIGITS	5-5	91.6	61.0	115.0	83.6	62.0	103.0
	5-20	92.7	75.0	133.0	91.2	80.5	126.5
	5-40	93.4	73.5	129.0	92.2	83.0	130.5
	40-40	93.6	66.0	121.0	94.0	95.8	143.6
SSC	3-3	73.7	57.0	89.0	73.3	75.0	96.0
	3-10	75.4	57.0	89.0	75.3	75.0	98.0
	3-20	75.5	58.0	88.0	77.5	76.0	99.0
	20-20	75.7	56.8	89.0	78.2	82.2	108.0

3.5. Reducing time delay in FS coding

As introduced in Section 2.2.2, CuBa-LiF neuron is used in the output layer for decision making and large time constants in the output layer lead to longer time delay in decision making. We therefore aim to reduce time delay of FS models by applying smaller τ_{m2} in output layer. To validate our intuition, we test different combination of τ_{m2} and τ_{s2} and compare them with FR coding, as outlined in Table 5.

Table 5 shows that a smaller τ_{m2} leads to a significant reduction of time delay for FS models, while this effect is not apparent for FR models, where the delay may even increase (see NTIDIGITS and SSC). Despite FS models still having a longer overall time delay compared to FR models, the gap between them is notably reduced. With the same τ_{m2} , decreasing τ_{s2} results in lower accuracy in FS models due to a shorter memory of previous stimuli. FR models also exhibit a similar trend but are more resilient to changes in τ_{m2} (see NTIDIGITS).

To summarize, using a combination of a small τ_{m2} and a large τ_{s2} can effectively reduce time delay in FS coding without compromising accuracy.

3.6. Effectiveness of adaptive D

We implement an experiment to validate the effectiveness of adaptive D for adjusting Gaussian window size in backpropagation, as introduced in Section 2.3.1. Results from Liu et al. (2023) have demonstrated that the value of D has a significant impact on accuracy. We carefully select fixed D values for different datasets to ensure optimal performance, as listed in Table 1. Table 6 show the comparison between fixed and adaptive D . Results indicate that using adaptive D achieves performance comparable to that of using an appropriately chosen fixed D .

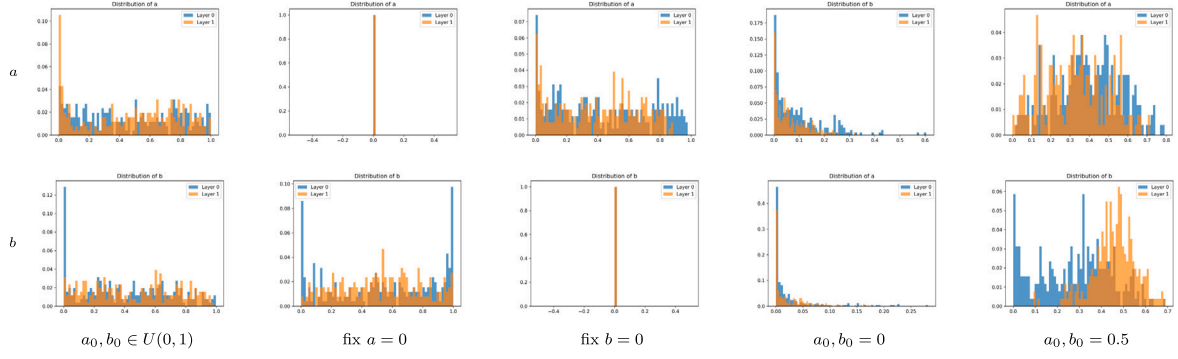


Fig. 8. The distribution of a and b after training (SHD dataset). Larger value of a and b leads to more intense oscillations.

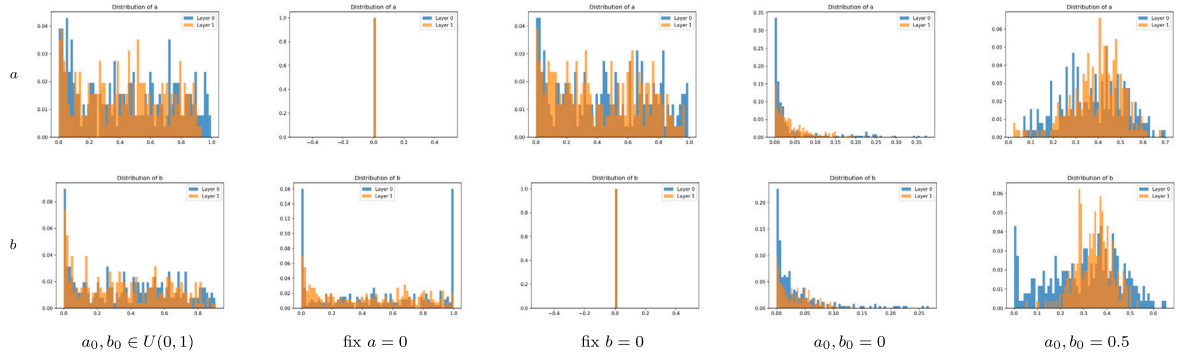


Fig. 9. The distribution of a and b after training (NTIDIGITS dataset). Larger value of a and b leads to more intense oscillations.

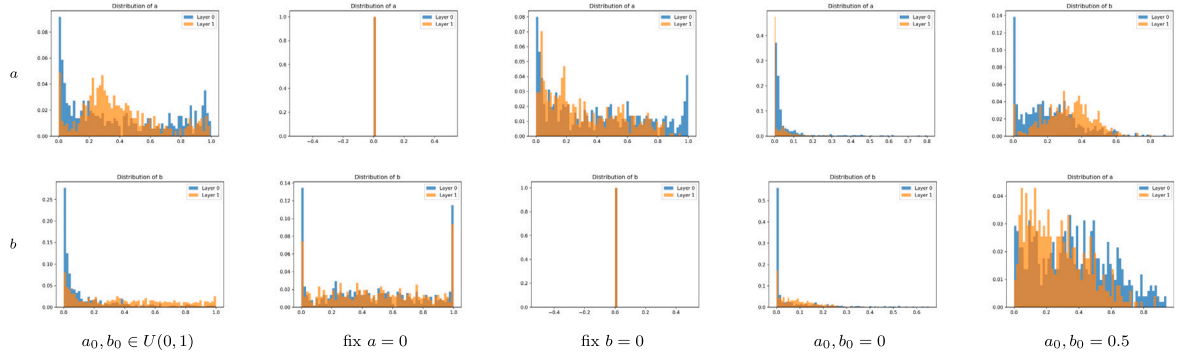


Fig. 10. The distribution of a and b after training (SSC dataset). Larger value of a and b leads to more intense oscillations.

Table 6

Comparison results of using fixed D (fix) and adaptive D (adapt) in backpropagation. **Bold** values represent the best results for the same dataset. Using adaptive D achieves performance comparable to that of using an appropriately chosen fixed D .

	D	Acc (%) $\uparrow$	$N_s \downarrow$	$t_d(50\%) \downarrow$	$t_d(90\%) \downarrow$
SHD	fix	94.2	7.1	56.2	71.0
	adapt.	93.7	6.1	52.5	65.5
NTIDIGITS	fix	93.3	4.8	92.0	137.0
	adapt.	94.0	5.0	95.8	143.6
SSC	fix	78.2	5.2	82.2	108.0
	adapt.	78.0	5.6	84.0	107.5

3.7. Ablation study for a and b

To validate the effectiveness of oscillations in feature extraction, we conduct experiments involving various initializations or training schemes for parameters a and b in the adaptation term. Numerical results are presented in Table 7, and the distributions of a and b across

neurons in the two hidden layers after training for SHD, NTIDIGITS, and SSC datasets are shown in Figs. 8 to 10. As discussed in Section 2.1, greater value of a and b results in more intense oscillations. We can observe that the learned distributions of a and b for SHD and NTIDIGITS are similar, with the parameters generally following Gaussian or uniform-like distributions, except for values near the borders. In contrast, the SSC distribution deviates from these patterns, which is difficult to explain given that these parameters are optimized for optimal performance.

By default, we initialize a_0 and b_0 from a uniform distribution $U(0, 1)$ and constrain their values to be between 0 and 1 during training, where a_0 and b_0 represent the initial values, as shown in row 1 of Table 7. Interestingly, we observed no significant degradation in performance when keeping the values of a and b fixed during training (compare rows 1 and 2 in Table 7). This suggests that introducing random fixed oscillation formats in the system leads to a significant improvement in performance except for SSC. In the first column in Figs. 8 to 10, the learned distributions of a and b still closely follow a uniform

Table 7

Comparison results of models with different initialization or training of a, b .

		SHD	NTIDIGITS	SSC
1	$a_0, b_0 \in U[0, 1]$	94.3	94.1	78.7
2	fix a, b	93.9	94.1	74.2
3	fix $a = 0$	88.1	91.1	73.1
4	fix $b = 0$	93.3	93.2	73.8
5	$a_0, b_0 = 0$	88.0	91.5	73.5
6	$a_0, b_0 = 0.5$	93.7	90.5	74.0

distribution, but with some values shifting towards 0, indicating that certain neural dynamics are less oscillatory after training.

From rows 3–6 in Table 7, it is evident that performance declines in all these scenarios compared to randomly initializing and training a and b . Notably, comparing rows 3 and 4, the accuracy experiences a more significant decline when a is set to 0 compared to when $b = 0$. This suggests that the oscillations introduced by a play a more crucial role in enhancing performance, which aligns with our intuition. The value of b induces oscillations only with the spike-triggered term and only influences the system when a spike occurs. Given that output spike trains are sparse, the oscillations introduced by b have a relatively limited impact.

In the case of initializing a and b with a constant value (see rows 5 and 6 in Table 7), setting them to 0.5 yields better results than using 0. As shown in the last two columns of the distribution figures, when a and b are initialized with 0, the learned values for a and b tend to be closer to 0. In this scenario, neurons exhibit fewer and shorter-term oscillations, resulting in decreased performance.

Overall, we can conclude that the oscillations in neural dynamics promote long-term temporal correlations and improve feature extraction, thereby enhancing overall accuracy. For easier datasets like SHD and NTIDIGITS, using fixed random oscillation formats provides performance comparable to that of trainable a and b , and parameter a has a more significant impact than b .

4. Conclusion

In this paper, we introduce a CuAdLIF neuron characterized by membrane potential adaptation and delayed responses to input stimuli. Using FS coding, we construct a hybrid system with CuAdLIF or other neuron models in hidden layers and CuBa-LIF neurons in the output layer. We improve FS coding by implementing a time constant initialization strategy to reduce output time delays and propose using adaptive Gaussian window sizes in backpropagation for adaptive training. In our experiments, we evaluate FS coding across different neuron models on classification problems using auditory datasets. Compared to other neuron types, the CuAdLIF neuron enhances temporal correlations and effectively extract long-term features, resulting in high accuracy for event-based data sequences with rich temporal structures. In addition, the CuAdLIF neuron enables fast responses to changing signals. The results also demonstrate the outstanding performance and adaptability of FS coding across various neuron types. Furthermore, our strategies effectively reduce the gap of time delays in decision making between FS and FR codings.

CRediT authorship contribution statement

Siying Liu: Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Pier Luigi Dragotti:** Writing – review & editing, Supervision, Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Siying Liu was supported by the scholarship from the Department of Electrical and Electronic Engineering, Imperial College London, UK.

Appendix. Stability analysis of the CuAdLIF neuron

A.1. Linear component of CuAdLIF neuron

Referring to the continuous-time dynamics of the CuAdLIF neuron in Eq. (1), and denoting the input signal as $X(t)$, the system becomes a linear system if no spike fires, which can be represented as follows:

$$\begin{aligned}\tau_s \frac{dI(t)}{dt} &= -I(t) + X(t) \\ \tau_m \frac{dU(t)}{dt} &= -U(t) + I(t) - W(t) \\ \tau_w \frac{dW(t)}{dt} &= -W(t) + aU(t).\end{aligned}\quad (\text{A.1})$$

This equation can be written as follows:

$$\frac{d\mathbf{Y}}{dt} = \mathbf{A}\mathbf{Y} + \mathbf{B}X(t), \quad (\text{A.2})$$

where $\mathbf{Y} = \begin{bmatrix} I(t) \\ U(t) \\ W(t) \end{bmatrix}$ is the state vector, $\mathbf{A} = \begin{bmatrix} -\frac{1}{\tau_s} & 0 & 0 \\ \frac{1}{\tau_m} & -\frac{1}{\tau_m} & -\frac{1}{\tau_m} \\ 0 & \frac{a}{\tau_w} & -\frac{1}{\tau_w} \end{bmatrix}$ is the

system matrix, and $\mathbf{B} = \begin{bmatrix} \frac{1}{\tau_s} \\ 0 \\ 0 \end{bmatrix}$ is the coefficient vector for input stimulus $X(t)$.

The eigenvalues of the system matrix $\mathbf{A}$ determines the stability and dynamics of the system. By solving $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$, these eigenvalues can be obtained as follows:

$$\begin{aligned}\lambda_1 &= \frac{-(\tau_m + \tau_w) + \sqrt{\tau_m^2 + \tau_w^2 - (4a + 2)\tau_m\tau_w}}{2\tau_m\tau_w} \\ \lambda_2 &= \frac{-(\tau_m + \tau_w) - \sqrt{\tau_m^2 + \tau_w^2 - (4a + 2)\tau_m\tau_w}}{2\tau_m\tau_w} \\ \lambda_3 &= -\frac{1}{\tau_s}.\end{aligned}\quad (\text{A.3})$$

For system stability, it is essential that the real part of all the eigenvalues is negative. Considering $\tau_s > 0$, λ_3 is thus always negative. Due to the condition $\tau_w \gg \tau_m > 0$ and the subtraction of the square root, λ_2 consistently has a negative real part, ensuring that it contributes to stability of the system. As for λ_1 , the expression under the square root, denoted as $O = \tau_m^2 + \tau_w^2 - (4a + 2)\tau_m\tau_w$, is influenced by the parameter a . The discussion of λ_1 is as follows:

- (1) If $O < 0$, λ_1 becomes a complex number with a negative real part. This requires the condition:

$$a > \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}. \quad (\text{A.4})$$

Therefore, the system exhibits stable oscillatory behavior. The oscillation frequency is determined by the imaginary part of the eigenvalue, i.e. $\omega = \frac{\sqrt{(4a+2)\tau_m\tau_w - \tau_m^2 - \tau_w^2}}{2\tau_m\tau_w}$. An increase in the value of a leads to a higher oscillation frequency ω .

- (2) When $O \geq 0$, meaning $a \leq \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}$, λ_1 is real but can be positive.

This occurs when $-(\tau_m + \tau_w) + \sqrt{O} > 0$, which holds for $a < 0$. Consequently, when $a < 0$, a positive λ_1 results in an unstable system. In contrast, when $0 \leq a \leq \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}$, λ_1 is a negative real value, leading to a stable system without oscillations.

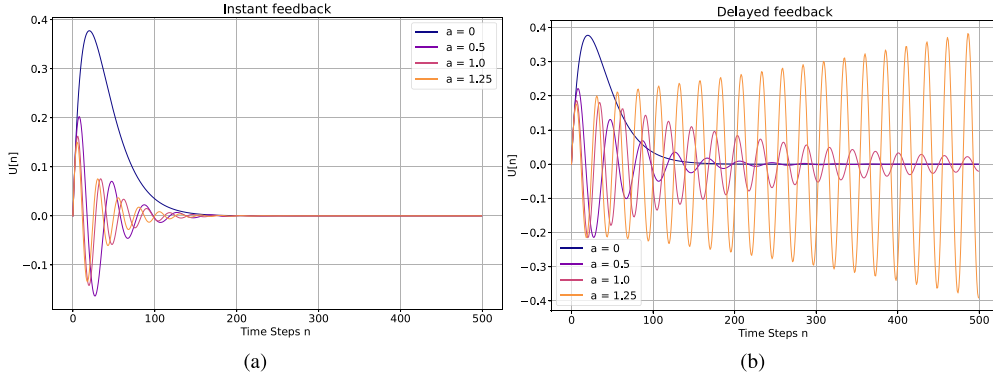


Fig. A.11. Dynamics of CuAdLIF neuron for varying values of a with (a) instant feedback and (b) delayed feedback across $N=500$ steps. The system constants are set to $\tau_m = \tau_s = 20$, $\tau_w = 100$, $\theta = 0.5$.

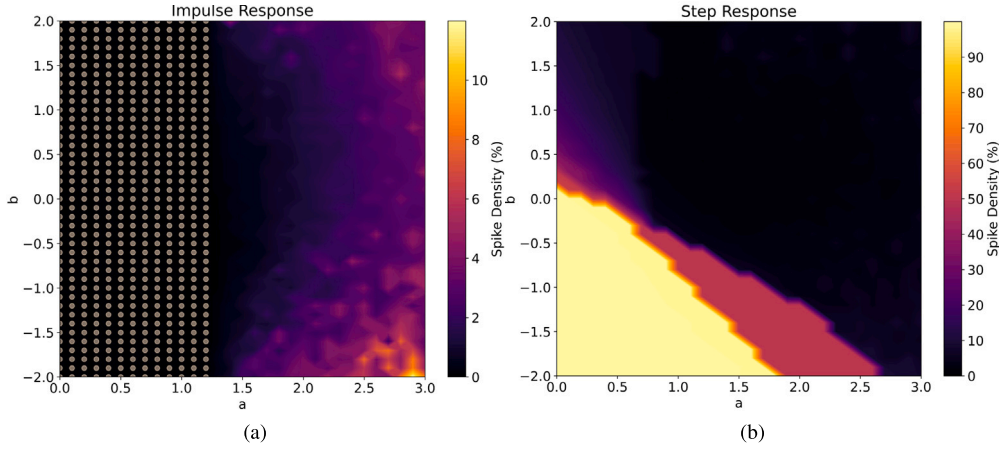


Fig. A.12. Spike density of CuAdLIF neuron for varying values of a and b , including (a) impulse response and (b) step response across $N=1000$ steps. Brown points indicate the zero spike density. The analysis spans parameters $a \in [0, 3]$ and $b \in [-2, 2]$, with the system constants set to $\tau_m = \tau_s = 20$, $\tau_w = 100$, $\theta = 0.5$.

In summary, the linear system is stable without oscillations when $0 \leq a \leq \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}$. It remains stable with oscillations for $a > \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}$, with the frequency of oscillations increasing with a . However, the system becomes unstable when $a < 0$, resulting in a positive eigenvalue.

A.2. Discretization, nonlinearity and delayed feedback

By employing the exponential decay method, the differential equation $\frac{dY(t)}{dt} = -Y(t) + X(t)$ can be discretized as follows:

$$Y[n] = e^{-\frac{\Delta t}{\tau}} Y[n-1] + (1 - e^{-\frac{\Delta t}{\tau}}) X[n]. \quad (\text{A.5})$$

This discretization allows us to obtain the discretized CuAdLIF neural dynamics as described in Eq. (3).

However, there are some adjustments in the discretization process compared to the continuous model. First, we omit the factor $(1 - \beta)$ in the term for $X[n]$, simplifying its influence. We also remove $(1 - \gamma)$ in the term for $W[n]$, meaning that the parameter a in Eq. (3) in the discrete system is equivalent to what is essentially $(1 - \gamma)a$ in the continuous framework. Given that $\gamma = e^{-\frac{\Delta t}{\tau_w}}$ approaches one for large values of τ_w , the key boundary $(1 - \gamma) \frac{(\tau_m - \tau_w)^2}{4\tau_m\tau_w}$ in Eq. (A.4) tends towards zero. Therefore, it is crucial to ensure that a remains non-negative ($a \geq 0$) to maintain system stability when no spike fires.

Another modification is the delayed feedback in the second equation to update membrane potential. To accurately reflect the dynamics of the continuous system described in Eq. (1), the update of the membrane potential should be $U[n] = \alpha U[n-1] + (1 - \alpha)(I[n-1] - W[n-1])$, where the synaptic current from the previous step is used as input.

However, the formula presented in Eq. (3) updates $U[n]$ based on $I[n]$ from the current timestep, thereby introducing an implicit “delay” or “memory effect” that is not present in the continuous model. This delayed feedback introduces complexities into the system behavior, such as oscillations and potential instabilities, making the analysis of stability more challenging.

Fig. A.11 illustrates the neural dynamics under instant feedback and the delayed feedback with parameter a varying from 0 to 1.25. The membrane potential $U[n]$ is updated using either $I[n-1]$ or $I[n]$. At $a = 0$, the system behaves like a CuBa-LIF neuron without oscillations. The plot in Fig. A.11(a) demonstrates that instant feedback induces stable oscillatory behavior with oscillation duration similar to that of the CuBa-LIF model, which is consistent with the linear analysis in Appendix A.1. In contrast, Fig. A.11(b) reveals that delayed feedback leads to more significant oscillations and takes longer to return to a stable state compared to the non-delayed system. In fact, it can cause instability when $a = 1.25$. This highlights the impact of feedback latency on neural dynamics. Long-term oscillations are beneficial for improving temporal correlations in the system. Nonetheless, it is crucial to carefully regulate the values of a and b to ensure stability.

Furthermore, the introduction of spike firing, especially in the form of nonlinearity and conditional logic through the threshold-based spike function $f(x)$, can induce significant changes in system stability and dynamics.

Due to the complex neural dynamics in the discretized model, we employ numerical analysis to investigate the system stability across various values of a and b . Specifically, we calculate the spike density in response to a unit impulse (with $X[1] = 1$ and $X[n] = 0$, $n \neq 1$) and a unit step input ($X[n] = 1$ for $n = 1, \dots, N$), where $N = 1000$. The

analysis covers parameters ranges $a \in [0, 3]$ and $b \in [-2, 2]$, with system constants $\tau_m = \tau_s = 20$, $\tau_w = 100$ and $\theta = 0.5$.

Fig. A.12 illustrates the variation in spike density for different combinations of a and b . In the scenario of the impulse response shown in Fig. A.12(a), a stable system is unlikely to produce any further spikes if the neuron fails to fire an early spike, leading to zero spike density (indicated by the brown points). It is observed that the spike density becomes non-zero once a exceeds approximately 1.5, suggesting a divergence in the membrane potential. Additionally, a higher absolute value of b increases the likelihood of system instability. For the step response scenario in Fig. A.12(b), negative values of b tend to induce intensive firing due to the persistent input signals.

Consequently, we limit the values of a and b within the range of $[0, 1]$ for both parameters to ensure the stability of the system.

Data availability

Data will be made available on request.

References

- Anumula, J., Neil, D., Delbruck, T., & Liu, S.-C. (2018). Feature Representations for Neuromorphic Audio Spike Streams. *Frontiers in Neuroscience*, 12, URL: <https://www.frontiersin.org/articles/10.3389/fnins.2018.00023>.
- Bellec, G., Salaj, D., Subramoney, A., Legenstein, R., & Maass, W. (2018). Long short-term memory and Learning-to-learn in networks of spiking neurons. In *Advances in neural information processing systems*, vol. 31. Curran Associates, Inc., URL: <https://proceedings.neurips.cc/paper/2018/hash/c203d8a151612ac12457e4d67635a95-Abstract.html>.
- Bittar, A., & Garner, P. N. (2022). A surrogate gradient spiking baseline for speech command recognition. *Frontiers in Neuroscience*, 16, URL: <https://www.frontiersin.org/articles/10.3389/fnins.2022.865897>.
- Brette, R. (2015). Philosophy of the spike: Rate-based vs. spike-based theories of the brain. *Frontiers in Systems Neuroscience*, 9, URL: <https://www.frontiersin.org/articles/10.3389/fnys.2015.00151>.
- Brunel, N., Hakim, V., & Richardson, M. J. E. (2003). Firing-rate resonance in a generalized integrate-and-fire neuron with subthreshold resonance. *Physical Review E*, 67, Article 051916. <https://doi.org/10.1103/PhysRevE.67.051916>, URL: <https://link.aps.org/doi/10.1103/PhysRevE.67.051916>.
- Chen, Y., Mai, Y., Feng, R., & Xiao, J. (2022). An adaptive threshold mechanism for accurate and efficient deep spiking convolutional neural networks. *Neurocomputing*, 469, 189–197. <https://doi.org/10.1016/j.neucom.2021.10.080>, URL: <https://www.sciencedirect.com/science/article/pii/S092523122101568X>.
- Comsa, I.-M., Potempa, K., Versari, L., Fischbacher, T., Gesmundo, A., & Alakuijala, J. (2021). Temporal coding in spiking neural networks with alpha synaptic function: Learning with backpropagation. *IEEE Transactions on Neural Networks and Learning Systems*, 1–14. <https://doi.org/10.1109/TNNLS.2021.3071976>, URL: <https://ieeexplore.ieee.org/document/9416238/>.
- Cramer, B., Stradmann, Y., Schemmel, J., & Zenke, F. (2022). The Heidelberg spiking datasets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 33, 2744–2757. <https://doi.org/10.1109/TNNLS.2020.3044364>, URL: <http://arxiv.org/abs/1910.07407>, arXiv:1910.07407 [cs, q-bio].
- Deckers, L., Van Damme, L., Van Leekwijck, W., Tsang, I. J., & Latré, S. (2024). Co-learning synaptic delays, weights and adaptation in spiking neural networks. *Frontiers in Neuroscience*, 18, <https://doi.org/10.3389/fnins.2024.1360300>, URL: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2024.1360300/full>.
- Engel, T. A., Schimansky-Geier, L., Herz, A., Schreiber, S., & Erchova, I. (2008). Subthreshold membrane-potential resonances shape spike-train patterns in the entorhinal cortex. *Journal of Neurophysiology*, 100, 1576–1589. <https://doi.org/10.1152/jn.01282.2007>, URL: <https://journals.physiology.org/doi/full/10.1152/jn.01282.2007>.
- Fang, W., Chen, Y., Ding, J., Yu, Z., Masquelier, T., Chen, D., Huang, L., Zhou, H., Li, G., & Tian, Y. (2023). SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9, eadi1480. <https://doi.org/10.1126/sciadv.adi1480>, URL: <https://www.science.org/doi/10.1126/sciadv.adi1480>.
- Fang, W., Yu, Z., Chen, Y., Masquelier, T., Huang, T., & Tian, Y. (2021). Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *2021 IEEE/CVF international conference on computer vision (ICCV)* (pp. 2641–2651). Montreal, QC, Canada: IEEE, <https://doi.org/10.1109/ICCV48922.2021.00266>, URL: <https://ieeexplore.ieee.org/document/9711070/>.
- Gerstner, W. (1995). Time structure of the activity in neural network models. *Physical Review E*, 51, 738–758. <https://doi.org/10.1103/PhysRevE.51.738>, URL: <https://link.aps.org/doi/10.1103/PhysRevE.51.738>.
- Göltz, J., Kriener, L., Baumbach, A., Billaudelle, S., Breitwieser, O., Cramer, B., Dold, D., Kungl, A. F., Senn, W., Schemmel, J., Meier, K., & Petrovici, M. A. (2021). Fast and energy-efficient neuromorphic deep learning with first-spike times. *Nature Machine Intelligence*, 3, 823–835. <https://doi.org/10.1038/s42256-021-00388-x>, URL: <https://www.nature.com/articles/s42256-021-00388-x>.
- Grimaldi, A., & Perrinet, L. U. (2023). Learning heterogeneous delays in a layer of spiking neurons for fast motion detection. *Biological Cybernetics*, 117, 373–387. <https://doi.org/10.1007/s00422-023-00975-8>.
- Hammouamri, I., Khalfaoui-Hassani, I., & Masquelier, T. (2023). Learning delays in spiking neural networks using dilated convolutions with learnable spacings. URL: <http://arxiv.org/abs/2306.17670>, arXiv:2306.17670 [cs].
- Hodgkin, A. L., & Huxley, A. F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117, 500–544. <https://doi.org/10.1113/jphysiol.1952.sp004764>, URL: <https://onlinelibrary.wiley.com/doi/abs/10.1113/jphysiol.1952.sp004764>.
- Hunsberger, E., & Eliasmith, C. (2015). Spiking deep networks with LIF neurons. <https://doi.org/10.48550/arXiv.1510.08829>, URL: <http://arxiv.org/abs/1510.08829>, arXiv:1510.08829 [cs].
- Izhikevich, E. M. (2001). Resonate-and-fire neurons. *Neural Networks*, 14, 883–894. [https://doi.org/10.1016/S0893-6080\(01\)00078-8](https://doi.org/10.1016/S0893-6080(01)00078-8), URL: <https://www.sciencedirect.com/science/article/pii/S0893608001000788>.
- Kheradpisheh, S. R., & Masquelier, T. (2020). Temporal backpropagation for spiking neural networks with one spike per neuron. *International Journal of Neural Systems*, 30, Article 2050027. <https://doi.org/10.1142/S0129065720500276>, URL: <https://www.worldscientific.com/doi/abs/10.1142/S0129065720500276>.
- Kiselev, M. (2016). Rate coding vs. temporal coding - is optimum between? In *2016 international joint conference on neural networks (IJCNN)* (pp. 1355–1359). <https://doi.org/10.1109/IJCNN.2016.7727355>.
- Li, G., Deng, L., Tang, H., Pan, G., Tian, Y., Roy, K., & Maass, W. (2023). Brain inspired computing: A systematic survey and future trends. <https://doi.org/10.36227/techrxiv.21837027.v1>, URL: https://www.techrxiv.org/articles/preprint/Brain_Inspired_Computing_A_Systematic_Survey_and_Future_Trends/21837027/1.
- Liu, S., Leung, V. C. H., & Dragotti, P. L. (2023). First-spike coding promotes accurate and efficient spiking neural networks for discrete events with rich temporal structures. *Frontiers in Neuroscience*, 17, URL: <https://www.frontiersin.org/articles/10.3389/fnins.2023.1266003>.
- Mirsadeghi, M., Shalchian, M., Kheradpisheh, S. R., & Masquelier, T. (2021). STDi-BP: Spike time displacement based error backpropagation in multilayer spiking neural networks. *Neurocomputing*, 427, 131–140. <https://doi.org/10.1016/j.neucom.2020.11.052>, URL: <https://www.sciencedirect.com/science/article/pii/S0925231220318452>.
- Mostafa, H. (2018). Supervised learning based on temporal coding in spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 29, 3227–3235. <https://doi.org/10.1109/TNNLS.2017.2726060>.
- Perez-Nieves, N., Leung, V. C. H., Dragotti, P. L., & Goodman, D. F. M. (2021). Neural heterogeneity promotes robust learning. *Nature Communications*, 12, 5791. <https://doi.org/10.1038/s41467-021-26022-3>, URL: <https://www.nature.com/articles/s41467-021-26022-3>.
- Pfeiffer, M., & Pfeil, T. (2018). Deep learning with spiking neurons: Opportunities and challenges. *Frontiers in Neuroscience*, 12, 774. <https://doi.org/10.3389/fnins.2018.00774>, URL: <https://www.frontiersin.org/article/10.3389/fnins.2018.00774/full>.
- Sakemi, Y., Yamamoto, K., Hosomi, T., & Aihara, K. (2023). Sparse-firing regularization methods for spiking neural networks with time-to-first-spike coding. *Scientific Reports*, 13, 22897. <https://doi.org/10.1038/s41598-023-50201-5>, URL: <https://www.nature.com/articles/s41598-023-50201-5>.
- Shaban, A., Bezugam, S. S., & Suri, M. (2021). An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation. *Nature Communications*, 12, 4234. <https://doi.org/10.1038/s41467-021-24427-8>, URL: <https://www.nature.com/articles/s41467-021-24427-8>.
- Shrestha, S. B., & Song, Q. (2017). Robust spike-train learning in spike-event based weight update. *Neural Networks*, 96, 33–46. <https://doi.org/10.1016/j.neunet.2017.08.010>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0893608017302009>.
- Shrestha, S. B., Zhu, L., & Sun, P. (2022). Spikemax: Spike-based loss methods for classification. URL: <http://arxiv.org/abs/2205.09845>, arXiv:2205.09845 [cs].
- Song, Z., Wu, J., Zhang, M., Shou, M. Z., & Li, H. (2024). Spiking-leaf: A learnable auditory front-end for spiking neural networks. In *ICASSP 2024 - 2024 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 226–230). <https://doi.org/10.1109/ICASSP48485.2024.10446789>, URL: <https://ieeexplore.ieee.org/abstract/document/10446789/figures#figures>.
- Sun, P., Eqilmi, E., Chua, Y., Devos, P., & Botteldooren, D. (2023). Adaptive axonal delays in feedforward spiking neural networks for accurate spoken word recognition. In *ICASSP 2023 - 2023 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 1–5). Rhodes Island, Greece: IEEE, <https://doi.org/10.1109/ICASSP49357.2023.10094768>, URL: <https://ieeexplore.ieee.org/document/10094768/>.

- Sun, P., Wu, J., Zhang, M., Devos, P., & Botteldooren, D. (2024). Delay learning based on temporal coding in spiking neural networks. *Neural Networks*, 180, Article 106678. <http://dx.doi.org/10.1016/j.neunet.2024.106678>, URL: <https://www.sciencedirect.com/science/article/pii/S0893608024006026>.
- Sun, P., Zhu, L., & Botteldooren, D. (2022). Axonal delay as a short-term memory for feed forward deep spiking neural networks. In *ICASSP 2022 - 2022 IEEE international conference on acoustics, speech and signal processing (ICASSP)* (pp. 8932–8936). Singapore, Singapore: IEEE, <http://dx.doi.org/10.1109/ICASSP43922.2022.9747411>, URL: <https://ieeexplore.ieee.org/document/9747411/>.
- Wu, Y., Deng, L., Li, G., Zhu, J., & Shi, L. (2018). Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12, 331. <http://dx.doi.org/10.3389/fnins.2018.00331>, URL: <https://www.frontiersin.org/article/10.3389/fnins.2018.00331/full>.
- Yao, M., Gao, H., Zhao, G., Wang, D., Lin, Y., Yang, Z., & Li, G. (2021). Temporal-wise attention spiking neural networks for event streams classification. In *2021 IEEE/CVF international conference on computer vision (ICCV)* (pp. 10201–10210). Montreal, QC, Canada: IEEE, <http://dx.doi.org/10.1109/ICCV48922.2021.01006>, URL: <https://ieeexplore.ieee.org/document/9711310/>.
- Yao, X., Li, F., Mo, Z., & Cheng, J. (2023). GLIF: A unified gated leaky integrate-and-fire neuron for spiking neural networks. <http://dx.doi.org/10.48550/arXiv.2210.13768>, URL: <http://arxiv.org/abs/2210.13768> [cs].
- Yao, M., Zhao, G., Zhang, H., Hu, Y., Deng, L., Tian, Y., Xu, B., & Li, G. (2023). Attention spiking neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1–18. <http://dx.doi.org/10.1109/TPAMI.2023.3241201>.
- Yin, B., Corradi, F., & Bohté, S. M. (2021). Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks. *Nature Machine Intelligence*, 3, 905–913. <http://dx.doi.org/10.1038/s42256-021-00397-w>, URL: <https://www.nature.com/articles/s42256-021-00397-w>.
- Yu, C., Gu, Z., Li, D., Wang, G., Wang, A., & Li, E. (2022). STSC-SNN: Spatio-temporal synaptic connection with temporal convolution and attention for spiking neural networks. *Frontiers in Neuroscience*, 16, Article 1079357. <http://dx.doi.org/10.3389/fnins.2022.1079357>, URL: <http://arxiv.org/abs/2210.05241>, arXiv:2210.05241 [cs, q-bio, stat].
- Zenke, F., & Ganguli, S. (2018). SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30, 1514–1541. http://dx.doi.org/10.1162/neco_a_01086, URL: <https://direct.mit.edu/neco/article/30/6/1514-1541/8378>.
- Zhang, W., & Li, P. (2019). Spike-train level backpropagation for training deep recurrent spiking neural networks. In *Advances in neural information processing systems*, vol. 32. Curran Associates, Inc., URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/f42a37d114a480b6b57b60ea9a14a9d2-Abstract.html.
- Zhang, S., Yang, Q., Ma, C., Wu, J., Li, H., & Tan, K. C. (2023). Long short-term memory with two-compartment spiking neuron. <http://dx.doi.org/10.48550/arXiv.2307.07231>, URL: <http://arxiv.org/abs/2307.07231>, arXiv:2307.07231 [cs].
- Zheng, H., Zheng, Z., Hu, R., Xiao, B., Wu, Y., Yu, F., Liu, X., Li, G., & Deng, L. (2024). Temporal dendritic heterogeneity incorporated with spiking neural networks for learning multi-timescale dynamics. *Nature Communications*, 15, 277. <http://dx.doi.org/10.1038/s41467-023-44614-z>, URL: <https://www.nature.com/articles/s41467-023-44614-z>.
- Zhou, S., Li, X., Chen, Y., Chandrasekaran, S. T., & Sanyal, A. (2021). Temporal-coded deep spiking neural network with easy training and robust performance. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35, 11143–11151. <http://dx.doi.org/10.1609/aaai.v35i12.17329>, URL: <https://ojs.aaai.org/index.php/AAAI/article/view/17329>.
- Zhou, C., Yu, L., Zhou, Z., Ma, Z., Zhang, H., Zhou, H., & Tian, Y. (2023). Spikingformer: Spike-driven residual learning for transformer-based spiking neural network. URL: <http://arxiv.org/abs/2304.11954>, arXiv:2304.11954 [cs].
- Zhou, Z., Zhu, Y., He, C., Wang, Y., Yan, S., Tian, Y., & Yuan, L. (2022). Spikformer: When spiking neural network meets transformer. URL: <http://arxiv.org/abs/2209.15425>, arXiv:2209.15425 [cs].
- Zhu, R.-J., Zhao, Q., Zhang, T., Deng, H., Duan, Y., Zhang, M., & Deng, L.-J. (2022). TCJA-SNN: Temporal-channel joint attention for spiking neural networks. URL: <http://arxiv.org/abs/2206.10177>, arXiv:2206.10177 [cs].